Job Board (Hacker News API, Load More)

- **Problem:** Build a job board that shows the latest job postings from the Hacker News API. Show 6 jobs at first; a "Load more jobs" button fetches and appends the next 6 until there are none left.
- **Tech stack:** React (Vite, JavaScript) *(default, none given)*
- **Time limit:** 60 minutes *(default, none given)*
- **Often asked at:** Uber, Airbnb, Stripe. **Why it matters:** it tests a *two-step fetch* (ids first, then details), batched parallel requests and client-side pagination, plus the async bugs that come with them: double clicks, races and partial failures.

PART A: Thought process

General steps for any machine coding task

1. **Clarify requirements**: what must work, what is out of scope. Write assumptions down.
2. **List interactions**: every user action and its expected result. This is your test checklist.
3. **Sketch components**: a small tree, one responsibility each.
4. **Design state**: minimal source of truth; derive everything else.
5. **Define data contracts**: data shape, API/mock shape, function signatures.
6. **Find the hard parts**: solve them on paper before typing.
7. **Build in layers**: a working skeleton first, then one feature at a time, keeping it runnable.
8. **Handle states and edge cases**: loading, empty, error, invalid input.
9. **Wrap up**: walk the checklist, explain trade-offs, list what you'd add.

Time budget for 60 minutes

Phase	Minutes
Requirements, questions, assumptions	5
Interactions, components, state, contracts	7
API layer + first page (ids then details)	12
Load more: paging, append, double-click guard	12
Loading / error / retry / empty / end-of-list states	10
Job card UI, links, date formatting, styling	9
Walk-through, edge cases, trade-offs	5

Think out loud

Say every decision and its reason as you make it: *"I'll fetch the id list once and keep it as a snapshot, so page 2 is always ids 6 to 11 even if a new job is posted while you read page 1."* The interviewer grades your reasoning as much as the result. If you are stuck, say what you are trying; silence looks like being lost.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Fetch the list of job ids from `GET /v0/jobstories.json`.
- Fetch the details of each job from `GET /v0/item/{id}.json`.
- Show the first **6** jobs on load.
- A **"Load more jobs"** button fetches the next 6 and **appends** them below the existing ones.
- Hide the button when every job has been shown.
- Each job shows: **title** (a link to the posting if it has a `url`), **poster** (`by`) and **posted date** (`time`).

Non-functional requirements

Concern	What it means here
Performance	Fetch the 6 details of a page **in parallel**, not one after another. Fetch the id list once only.
Scalability	Paging is a slice over the id list, so 30 or 3000 ids cost the same per click.
Error handling	A failed page shows a message and a **Retry** that re-fetches the same page; already-loaded jobs stay.
Accessibility	Real `<button>` and `<a>` elements, `role="status"` for loading, `role="alert"` for errors, `aria-busy` on the list, visible focus ring.
Usability	Button shows "Loading…" and is disabled while a page is in flight; external links open in a new tab safely.
Reusability	API calls in one module, paging logic in pure functions, fetching in one hook; the card knows nothing about fetching.
Extensibility	Changing page size, adding infinite scroll or a filter touches the hook or a constant, not the components.

Clarifying questions to ask the interviewer

1. Page size: is it fixed at 6, and is the first load the same size as later loads?
2. Should it be a button, or infinite scroll as the user reaches the bottom?
3. What if one job in a page fails to load: show the other five, or treat the page as failed?
4. Jobs without a `url` (they have `text` instead): link to the HN discussion page, or show the text inline?

5. Should new jobs posted while the user is browsing appear, or is a stable snapshot fine?
6. Any date format, and do we need a "posted 3 days ago" relative time?

Assumptions we will build with

- Page size is **6** for every page.
- A **button**, not infinite scroll (easy to add later with an `IntersectionObserver`).
- A page is **all or nothing**: if any of its 6 requests fails, the page fails and Retry fetches all 6 again.
- Jobs with no `url` link to `https://news.ycombinator.com/item?id={id}`.
- The id list is fetched **once** and used as a snapshot for the whole session.
- Deleted or dead items (`null`, `deleted: true`, `dead: true`) are skipped.
- Date is shown as an absolute local date and time ("Oct 30, 2026, 12:30 PM").

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page opens	"Loading jobs…" shows; ids are fetched, then 6 details in parallel; 6 cards appear.
Click "Load more jobs"	Button shows "Loading…" and is disabled; 6 more cards are **appended**; earlier cards stay.
Click "Load more" 3 times very fast	Only **one** next page is requested; no skipped pages, no duplicates.
Last page loads	Fewer than 6 cards may arrive; the button disappears.
A request fails on first load	Error alert with Retry; no button.
A request fails on "Load more"	Error alert with Retry; loaded cards stay visible; button hidden until retry succeeds.
Click Retry	Same page is fetched again; on success the alert goes away and cards appear once.
Click a job title	Opens the job `url` (or the HN item page) in a new tab with `noopener`.
API returns an empty id list	"No jobs posted right now."
An id resolves to `null` / deleted	That job is skipped; the rest of the page renders.
Component unmounts mid-fetch	Requests are aborted; no state update after unmount.

This table is the implementation and test checklist.

3. Design the architecture

Module	Responsibility
`api/hn.js`	The only place that knows the HN URLs: `fetchJobIds`, `fetchJob`, error on non-2xx.

Module	Responsibility
`utils/jobs.js`	Pure helpers: `getPageIds`, `hasMorePages`, `isVisibleJob`, `getJobLink`, `getHost`, `formatDate`.
`hooks/useJobBoard.js`	Owns fetching and paging state; exposes `{ jobs, status, error, hasMore, loadMore, retry }`.
`components/JobCard.jsx`	Renders one job: title link, host, poster, date. No fetching.
`App.jsx`	Layout and state-driven UI: loading text, list, error with retry, empty text, load-more button.

```
App
├── useJobBoard()  ──>  api/hn.js  ──>  Hacker News API
│        └── utils/jobs.js (paging, filtering)
└── <ul>
     └── JobCard × n  ──> utils/jobs.js (formatting)
```

Start simple: the first version can have everything in `App`. Split the hook out as soon as the effect, abort and paging logic appear, because that is the part worth testing and reusing.

4. Design the state / data model

State or field	Purpose
`idsRef` (ref)	Snapshot of all job ids, fetched once. A ref because changing it should not re-run the effect.
`total`	Number of ids, `null` until known. Drives `hasMore`.
`pages`	`pages[n]` = visible jobs of page n. Written by index, so writes are idempotent.
`page`	The last requested page. Changing it triggers the fetch effect.
`attempt`	Retry counter; bumping it re-runs the effect for the same `page`.
`status`	`'loading' | 'success' | 'error'`. One field instead of `isLoading` + `isError` booleans that can contradict.
`error`	Message for the alert.

```
const idsRef = useRef(null);
const [total, setTotal] = useState(null);
const [pages, setPages] = useState([]);
const [page, setPage] = useState(0);
const [attempt, setAttempt] = useState(0);
const [status, setStatus] = useState('loading');
const [error, setError] = useState(null);

// derived, never stored
```

```
const jobs = pages.flat();
const hasMore = hasMorePages(total, page);
```

Senior note. `jobs` and `hasMore` are **derived**: storing them would be two more things to keep in sync. The cursor is also derived (`page * PAGE_SIZE`), so there is no separate offset to drift. The hook **owns** all fetching state; `App` only reads it and calls `loadMore` / `retry` ; `JobCard` owns nothing. The id list lives in a ref because it is a cache, not something the UI renders.

5. Design the data structures and contracts

Example data

```
// GET https://hacker-news.firebaseio.com/v0/jobstories.json
[50300643, 50280824, 50276779, 50179455, 50164991, 50155172, 50132839 /* ... */]

// GET https://hacker-news.firebaseio.com/v0/item/50300643.json
{
  "id": 50300643,
  "by": "syrusakbary",
  "time": 1793343635,            // unix SECONDS, not ms
  "title": "Wasmer (YC S19) Is Hiring OS/Hypervisor Experts in Europe",
  "type": "job",
  "url": "https://www.workatastartup.com/jobs/98722"   // optional; some jobs have "text" instead
}
```

The HN `id` is stable and unique, so it is the React `key` . Ids come newest first, so slicing gives newest-first pages for free.

API / interface contracts

Function	Input	Output
`fetchJobIds(signal)`	`AbortSignal`	`Promise<number[]>` ; rejects on network error or non-2xx
`fetchJob(id, signal)`	job id, `AbortSignal`	`Promise<Job | null>`
`getPageIds(ids, page, size = 6)`	ids, page index	ids of that page (`[]` past the end)
`hasMorePages(total, page, size = 6)`	total or `null` , last page	`true` if another page exists
`isVisibleJob(item)`	item or `null`	`false` for `null` , `deleted` , `dead`
`useJobBoard()`	none	`{ jobs, status, error, hasMore, loadMore, retry }`

6. Trace the data flow

Key interaction: the user clicks **"Load more jobs"** on page 0.

```
Click "Load more jobs"
      │
      ▼
loadMore(): status === 'success' && hasMore
      │   next = page + 1  (= 1)
      ▼
setPage(p => Math.max(p, 1))           ← state change
      │
      ▼
Re-render: page = 1 → effect [page, attempt] runs
      │   setStatus('loading') → button "Loading…", disabled
      ▼
idsRef already filled → getPageIds(ids, 1) → ids 6..11
      │
      ▼
Promise.all(6 × fetchJob(id, signal))  ← side effect, in parallel
      │
      ▼
filter(isVisibleJob) → setPages(next[1] = items) → setStatus('success')
      │
      ▼
Re-render: jobs = pages.flat() → 12 cards; hasMore recomputed → button back or gone
```

7. Important technical challenges

Challenge 1: Two-step fetch, done in parallel and only once

Problem. The ids endpoint gives only numbers. Fetching 6 details one after another with `await` in a loop takes 6 round trips (6 × 300 ms ≈ 1.8 s). Fetching the id list again on every "Load more" wastes a request and, worse, the list can change in between.

Solution. Fetch ids once into a ref. For each page, start all 6 detail requests at once with `Promise.all`, which also keeps the result order equal to the id order.

```
if (!idsRef.current) {
  idsRef.current = await fetchJobIds(signal);
  setTotal(idsRef.current.length);
}
const pageIds = getPageIds(idsRef.current, page);
const items = await Promise.all(pageIds.map((id) => fetchJob(id, signal)));
```

Challenge 2: Fast double clicks skip pages

Problem. The user clicks "Load more" three times before React re-renders. All three handlers see the same `status = 'success'` and `page = 0`. With `setPage(p => p + 1)` the page jumps to 3: pages 1 and 2 are never fetched and `pages` has holes. (This really happened while testing this solution: the list crashed on `undefined.id`.)

Solution. Each click asks for "the page after the one I saw", and `Math.max` collapses repeated asks into one. The button is also `disabled` while loading, which covers normal clicks after the re-render.

```
const loadMore = () => {
  if (status !== 'success' || !hasMore) return;
  const next = page + 1;
  setPage((p) => Math.max(p, next)); // 3 clicks in one tick → still page 1
};
```

Challenge 3: Races, unmounts and StrictMode double effects

Problem. In dev, React StrictMode runs every effect twice (mount, cleanup, mount). If both runs append their results, page 0 shows 12 cards with duplicates. A slow response for an old request could also land after the component unmounts.

Solution. Two guards. (1) An `AbortController` per effect run, aborted in cleanup; after each `await`, bail out if aborted. (2) Write results **by page index**, not by push, so even if the same page lands twice the state is the same.

```
useEffect(() => {
  const controller = new AbortController();
  load(controller.signal);
  return () => controller.abort();
}, [page, attempt]);

// inside load(), after the await
if (signal.aborted) return;
setPages((prev) => {
  const next = [...prev];
  next[page] = items.filter(isVisibleJob); // idempotent write
  return next;
});
```

Challenge 4: Failures without losing what is already loaded

Problem. Page 2 fails with a 500. If the code just sets an error and clears the list, the user loses 12 jobs they were reading. If it advances the page anyway, those 6 jobs are lost forever.

Solution. `fetch` does not reject on HTTP errors, so check `res.ok` and throw. On error keep `pages` as is, keep `page` unchanged and show Retry. Retry bumps `attempt`, which re-runs the effect for the **same** page.

```
async function getJSON(path, signal) {
  const res = await fetch(`${BASE_URL}/${path}.json`, { signal });
  if (!res.ok) throw new Error(`Request failed (${res.status})`);
  return res.json();
}

const retry = () => setAttempt((a) => a + 1); // same page, fresh attempt
```

8. Implement in the correct order

#	Step	Working result
1	Vite app, `App` with a heading	Page renders
2	`api/hn.js` with `fetchJobIds`, `fetchJob`, `res.ok` check	Can log ids and one item in the console
3	Effect in `App`: ids, then first 6 details with `Promise.all`	6 raw titles on screen
4	`JobCard` with link, host, poster, date	Cards look right
5	`page` state + `getPageIds`; "Load more" increments `page`	Pages append
6	`hasMorePages`; hide button at the end	No button after the last job
7	`status`: first-load text, "Loading..." + disabled button	No double requests on normal clicks
8	`AbortController` + `signal.aborted` checks, idempotent `pages[page]` write	No duplicates in StrictMode, no updates after unmount
9	`Math.max` in `loadMore`	Fast triple click loads one page
10	Error alert + `attempt` retry	Failed page can be retried, old jobs stay
11	Move logic into `useJobBoard`, helpers into `utils/jobs.js`	Clean structure, testable helpers
12	Empty state, `isVisibleJob` filter, a11y roles, styling	Every state covered

9. Cover all states and edge cases

State / case	What the user sees	How
Initial (first load)	"Loading jobs..." (`role="status"`)	`status === 'loading' && jobs.length === 0`
Loading more	Cards stay, button "Loading..." and disabled, list `aria-busy`	`status === 'loading'`
Success	Cards + "Load more jobs"	`status === 'success' && hasMore`
End of list	Cards, no button	`hasMorePages` false

State / case	What the user sees	How
Last page shorter than 6	Remaining 1–5 cards	`slice` past the end is safe
Empty (0 ids)	"No jobs posted right now."	`total === 0`
Error on first load	Alert + Retry, no button	`status === 'error'`
Error on load more	Old cards + alert + Retry	`pages` kept, `page` unchanged
HTTP 4xx/5xx	Treated as error	`res.ok` check
Deleted / dead / `null` item	Skipped	`isVisibleJob`
Job without `url`	Title links to HN item page	`getJobLink`
Invalid `url`	No host shown, link still works	`getHost` try/catch
`time` in seconds	Correct date, not 1970	`time * 1000`
Triple click in one tick	One page fetched	`Math.max(p, next)`
StrictMode double effect	No duplicate cards	abort + index write
Unmount mid-fetch	No warning, no update	`controller.abort()` in cleanup
New jobs posted while browsing	Pages stay stable	id snapshot in `idsRef`

10. Final structure

```
_64_job_board_hacker_news/
├── index.html
├── package.json
└── src/
    ├── main.jsx
    ├── App.jsx
    ├── App.css
    ├── api/
    │   └── hn.js
    ├── hooks/
    │   └── useJobBoard.js
    ├── utils/
    │   └── jobs.js
    └── components/
        └── JobCard.jsx
```

11. Full working code

`src/api/hn.js`

```
const BASE_URL = 'https://hacker-news.firebaseio.com/v0';

async function getJSON(path, signal) {
  const res = await fetch(`${BASE_URL}/${path}.json`, { signal });
  if (!res.ok) throw new Error(`Request failed (${res.status})`);
  return res.json();
}

// -> number[] (newest first, up to ~200 ids)
export const fetchJobIds = (signal) => getJSON('jobstories', signal);

// -> { id, by, time, title, url?, text?, type: 'job' } | null (deleted item)
export const fetchJob = (id, signal) => getJSON(`item/${id}`, signal);
```

`src/utils/jobs.js`

```
export const PAGE_SIZE = 6;

export function getPageIds(ids, page, size = PAGE_SIZE) {
  return ids.slice(page * size, (page + 1) * size);
}

// total is null until the id list arrives: assume there is more.
export function hasMorePages(total, page, size = PAGE_SIZE) {
  return total === null || (page + 1) * size < total;
}

// HN returns null for missing items and flags removed ones.
export function isVisibleJob(item) {
  return item != null && !item.deleted && !item.dead;
}

export function getJobLink(job) {
  return job.url ?? `https://news.ycombinator.com/item?id=${job.id}`;
}

export function getHost(url) {
  try {
    return new URL(url).hostname.replace(/^www\./, '');
  } catch {
```

```
    return null;
  }
}

export function formatDate(unixSeconds) {
  return new Date(unixSeconds * 1000).toLocaleString('en-US', {
    dateStyle: 'medium',
    timeStyle: 'short',
  });
}
```

src/hooks/useJobBoard.js

```
import { useEffect, useRef, useState } from 'react';
import { fetchJob, fetchJobIds } from '../api/hn';
import { getPageIds, hasMorePages, isVisibleJob } from '../utils/jobs';

export function useJobBoard() {
  const idsRef = useRef(null); // snapshot of ids, fetched once so pages stay stable
  const [total, setTotal] = useState(null);
  const [pages, setPages] = useState([]); // pages[n] = jobs of page n
  const [page, setPage] = useState(0); // last requested page
  const [attempt, setAttempt] = useState(0); // bump to retry the same page
  const [status, setStatus] = useState('loading'); // 'loading' | 'success' | 'error'
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();
    const { signal } = controller;

    async function load() {
      setStatus('loading');
      setError(null);
      try {
        if (!idsRef.current) {
          idsRef.current = await fetchJobIds(signal);
          setTotal(idsRef.current.length);
        }
        const pageIds = getPageIds(idsRef.current, page);
        const items = await Promise.all(pageIds.map((id) => fetchJob(id, signal)));
        if (signal.aborted) return;
        // Write by index, not push: running the same page twice can't duplicate.
        setPages((prev) => {
          const next = [...prev];
          next[page] = items.filter(isVisibleJob);
```

```
          return next;
        });
        setStatus('success');
      } catch (err) {
        if (signal.aborted) return;
        setError(err.message || 'Something went wrong');
        setStatus('error');
      }
    }

    load();
    return () => controller.abort();
  }, [page, attempt]);

  const jobs = pages.flat();
  const hasMore = hasMorePages(total, page);

  const loadMore = () => {
    if (status !== 'success' || !hasMore) return;
    // Clicks before the next render all see the same `page`: max() makes them one request.
    const next = page + 1;
    setPage((p) => Math.max(p, next));
  };
  const retry = () => setAttempt((a) => a + 1);

  return { jobs, status, error, hasMore, loadMore, retry };
}
```

src/components/JobCard.jsx

```
import { formatDate, getHost, getJobLink } from '../utils/jobs';

export default function JobCard({ job }) {
  const host = job.url ? getHost(job.url) : null;

  return (
    <li className="job-card">
      <h2 className="job-title">
        <a href={getJobLink(job)} target="_blank" rel="noopener noreferrer">
          {job.title}
        </a>
        {host && <span className="job-host"> ({host})</span>}
      </h2>
      <p className="job-meta">
        By {job.by} ·{' '}
```

```
        <time dateTime={new Date(job.time * 1000).toISOString()}>{formatDate(job.time)}</time>
      </p>
    </li>
  );
}
```

src/App.jsx

```
import JobCard from './components/JobCard';
import { useJobBoard } from './hooks/useJobBoard';
import './App.css';

export default function App() {
  const { jobs, status, error, hasMore, loadMore, retry } = useJobBoard();
  const isLoading = status === 'loading';
  const firstLoad = isLoading && jobs.length === 0;

  return (
    <main className="board">
      <h1>Hacker News Jobs</h1>

      {firstLoad && <p role="status" className="info">Loading jobs…</p>}

      {jobs.length > 0 && (
        <ul className="job-list" aria-busy={isLoading}>
          {jobs.map((job) => (
            <JobCard key={job.id} job={job} />
          ))}
        </ul>
      )}

      {status === 'success' && jobs.length === 0 && !hasMore && (
        <p className="info">No jobs posted right now.</p>
      )}

      {status === 'error' && (
        <div role="alert" className="error">
          <span>Couldn't load jobs: {error}</span>
          <button type="button" onClick={retry}>Retry</button>
        </div>
      )}

      {hasMore && !firstLoad && status !== 'error' && (
        <button type="button" className="load-more" onClick={loadMore} disabled={isLoading}>
          {isLoading ? 'Loading…' : 'Load more jobs'}
```

```
        </button>
      )}
    </main>
  );
}
```

src/App.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: #f6f6ef; color: #222; }

.board { max-width: 720px; margin: 0 auto; padding: 24px 16px; }
.board h1 { color: #ff6600; font-size: 1.75rem; margin: 0 0 16px; }

.job-list { list-style: none; margin: 0; padding: 0; display: grid; gap: 12px; }
.job-card { background: #fff; border: 1px solid #e3e3d8; border-radius: 8px; padding: 14px 16px; }
.job-title { font-size: 1rem; margin: 0 0 6px; line-height: 1.4; }
.job-title a { color: inherit; text-decoration: none; }
.job-title a:hover, .job-title a:focus-visible { text-decoration: underline; }
.job-host { color: #828282; font-weight: 400; font-size: 0.875rem; }
.job-meta { margin: 0; color: #666; font-size: 0.875rem; }

.load-more, .error button {
  margin-top: 16px; padding: 10px 18px; border: 0; border-radius: 6px;
  background: #ff6600; color: #fff; font-size: 0.95rem; cursor: pointer;
}
.load-more:disabled { opacity: 0.6; cursor: progress; }
.load-more:focus-visible, .error button:focus-visible { outline: 3px solid #222; outline-offset: 2px;
}

.info { color: #666; }
.error { margin-top: 16px; padding: 12px 16px; border-radius: 8px; background: #fde8e8; color:
#9b1c1c;
  display: flex; align-items: center; justify-content: space-between; gap: 12px; flex-wrap: wrap; }
.error button { margin-top: 0; }
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
```

```
    <App />
  </StrictMode>
);
```

Quick self-check of the pure helpers (`node selfcheck.mjs`)

```
import assert from 'node:assert/strict';
import { getPageIds, hasMorePages, isVisibleJob, getHost, getJobLink } from './src/utils/jobs.js';

const ids = Array.from({ length: 14 }, (_, i) => i + 1);
assert.deepEqual(getPageIds(ids, 0), [1, 2, 3, 4, 5, 6]);
assert.deepEqual(getPageIds(ids, 2), [13, 14]);
assert.deepEqual(getPageIds(ids, 3), []);
assert.equal(hasMorePages(null, 0), true);
assert.equal(hasMorePages(14, 1), true);
assert.equal(hasMorePages(14, 2), false);
assert.equal(hasMorePages(12, 1), false);
assert.equal(hasMorePages(0, 0), false);
assert.equal(isVisibleJob(null), false);
assert.equal(isVisibleJob({ id: 1, deleted: true }), false);
assert.equal(isVisibleJob({ id: 1, dead: true }), false);
assert.equal(isVisibleJob({ id: 1, title: 'x' }), true);
assert.equal(getHost('https://www.ycombinator.com/jobs'), 'ycombinator.com');
assert.equal(getHost('not a url'), null);
assert.equal(getJobLink({ id: 7 }), 'https://news.ycombinator.com/item?id=7');
console.log('selfcheck ok');
```

12. Wrap-up

Edge cases handled

- Parallel detail fetch per page; id list fetched once and kept as a stable snapshot.
- Fast multiple clicks load exactly one page; button disabled while loading.
- StrictMode double effects and unmounts: aborted requests, idempotent page writes, no duplicates.
- HTTP errors detected via `res.ok`; failed page retried in place; loaded jobs never lost.
- Deleted / dead / `null` items skipped; jobs without `url` link to HN; bad URLs don't crash the host label.
- Empty list, short last page and end of list each have their own UI.

Trade-offs to mention to the interviewer

- **All-or-nothing pages** (`Promise.all`) vs. **partial pages** (`Promise.allSettled`). All-or-nothing keeps paging simple and retry exact; partial shows more data but needs per-item retry and a way to fill gaps.
- **Snapshot of ids** vs. **live list**. A snapshot gives stable pages; new jobs only show after a reload. A live list needs de-duplication by id and a "N new jobs" banner.

- **Effect driven by `page`** vs. **an imperative `loadMore` async function**. The effect gets abort and cleanup for free and survives StrictMode; the imperative version needs its own in-flight ref.
- **Hand-rolled hook** vs. **TanStack Query `useInfiniteQuery`**. A library gives caching, retries and dedupe; in an interview the hand-rolled version shows you understand what it does.
- **Button** vs. **infinite scroll**. The button is accessible and predictable; infinite scroll hides the footer and needs a sentinel and focus care.

What to add with more time

- Tests: React Testing Library with a mocked `fetch` for first load, load more, triple click, error + retry, empty list.
- Skeleton cards while loading instead of plain text, to avoid layout jump.
- Move focus to the first new card after "Load more" (or announce "6 more jobs loaded" in a live region).
- Per-request timeout via `AbortSignal.any([signal, AbortSignal.timeout(8000)])` and automatic retry with backoff.
- Cache pages in `sessionStorage` so Back navigation restores the list and scroll position.
- Parse titles like "Company (YC S19) Is Hiring X" into company / batch / role for nicer cards and filters.
- Virtualize the list if it grows to thousands of rows.

13. Counter questions the interviewer may ask

Requirements

Q1. Why fetch the id list only once? What if new jobs are posted?

Fetching once gives a stable snapshot: page 2 is always ids 6–11, so nothing shifts or repeats. If new jobs matter, poll `jobstories` in the background, diff by id, and show a "3 new jobs, show them" banner that prepends them, instead of silently changing the pages under the user.

Q2. What if the interviewer wants infinite scroll instead of a button?

Keep the hook as is and swap the trigger: put a sentinel `<div>` after the list and call `loadMore()` when an `IntersectionObserver` reports it visible. `loadMore` already ignores calls while loading, so repeated intersections are safe. Keep a button as fallback for keyboard users.

Q3. What if one of the 6 item requests fails?

With `Promise.all` the whole page fails and Retry fetches all 6 again: simple and exact. If the product wants partial pages, use `Promise.allSettled`, render the fulfilled ones and track failed ids for a per-item retry. The cost is more state and gaps in order.

State design

Q4. Why `status` as one string instead of `isLoading` and `isError`?

Two booleans allow four combinations, and "loading and error at the same time" is meaningless. One field makes impossible states impossible and the JSX reads as a simple switch.

Q5. Why store `pages` as an array of arrays instead of a flat `jobs` array?

Writing `pages[page] = items` is idempotent: the same page landing twice gives the same state. Appending with `[...prev, ...items]` would duplicate cards in that case. `jobs` is then just `pages.flat()` at render time.

Q6. Why is the id list in a `useRef` and not `useState`?

It is a cache read by the effect, never rendered. If it were state in the effect's dependency list, setting it would re-run the effect and fetch the page again. Only `total` (which the UI needs for `hasMore`) is state.

Q7. Isn't `jobs = pages.flat()` on every render wasteful?

It's at most a few hundred items: microseconds. `useMemo` would add code for no visible gain. If the list became huge I'd memoize it, but at that size virtualization is the real fix.

Challenge: parallel fetching

Q8. Why `Promise.all` and not a `for … await` loop?

The loop waits for each request before starting the next: 6 round trips in series. `Promise.all` starts all 6 at once, so the page takes about as long as the slowest single request. It also returns results in input order, so the list stays newest first.

Q9. What if the page size were 100? Would you fire 100 requests at once?

Browsers cap connections per host (about 6 on HTTP/1.1; HTTP/2 multiplexes), and it's unkind to the API. I'd limit concurrency to around 6–10 with a small pool (or `p-limit`) and render items as they arrive, or ask for a batch endpoint on the backend.

Challenge: double clicks and races

Q10. The button is disabled while loading. Why do you also need `Math.max`?

`disabled` only takes effect after React re-renders. Clicks that land before that re-render (fast double-click, automated clicks) all run with the old `page`. `Math.max(p, page + 1)` makes them all ask for the same page, so they collapse into one. I found this bug while testing: three clicks skipped two pages.

Q11. Why not a `useRef` "inFlight" flag in an imperative `loadMore`?

It works, but you must reset it carefully on abort and unmount, and with StrictMode the cleanup can leave it stuck `true`. Driving the fetch from an effect on `page` gets cancellation and cleanup from React for free; the ref approach is the fallback if you can't use effects (e.g. outside React).

Q12. What does the `AbortController` protect against?

Three things: updates after unmount, StrictMode's first (thrown-away) effect run, and an old request finishing after a newer one has started. Aborting also cancels the network request itself, saving bandwidth. The `signal.aborted` check after `await` covers responses that resolved just before the abort.

Implementation steps

Q13. HN `time` is `1793343635`. Why multiply by 1000?

HN uses Unix **seconds**; JavaScript `Date` takes **milliseconds**. Without `* 1000` every job shows January 1970. I also put an ISO string in `<time dateTime>` so it's machine-readable.

Q14. Why `rel="noopener noreferrer"` on the job links?

With `target="_blank"` the new page could access `window.opener` and redirect this tab (tab-nabbing). Modern browsers imply `noopener`, but writing it is explicit and covers older ones; `noreferrer` also hides our URL from the job site.

Q15. How do you handle items that come back `null` or `deleted`?

`isVisibleJob` filters out `null`, `deleted` and `dead` items before they reach state. `hasMore` is based on ids consumed, not cards shown, so a page with a removed job still advances correctly and the button doesn't vanish early.

Performance and scaling

Q16. What if the API is slow (3–5 seconds per page)?

Show skeleton cards so the layout doesn't jump, keep earlier cards usable, and add a timeout with `AbortSignal.timeout` so a hung request turns into a retryable error. Prefetching the next page in the background right after the current one loads makes "Load more" feel instant.

Q17. What if there were 1 million jobs?

Paging by slice is still cheap, but holding 1 million ids client-side is not, and rendering thousands of cards slows scrolling. I'd ask for server-side cursor pagination (`?after=id&limit=20`) and virtualize the list (e.g. `react-window`) so only visible rows are in the DOM.

Q18. How would you cache so Back navigation doesn't refetch everything?

Store `{ ids, pages, page }` in `sessionStorage` (or a module-level cache, or TanStack Query) and hydrate from it on mount, then restore scroll position. Add a timestamp so stale caches older than a few minutes are dropped.

Trade-offs

Q19. Why not use TanStack Query's `useInfiniteQuery`?

In production I would: it handles caching, dedupe, retries, refetch on focus and the double-click problem. In an interview the point is to show I understand those problems, so I hand-roll it first and mention the library as the upgrade.

Q20. How would you test this?

Unit-test the pure helpers (`getPageIds` , `hasMorePages` , `isVisibleJob`) with plain asserts. With React Testing Library and a mocked `fetch` : first load shows 6, load more shows 12, triple click still shows 12, a 500 shows the alert and keeps cards, Retry recovers, 0 ids shows the empty text, and the button disappears at the end.